Sprint 6 · Reto «Datos sucios, cliente impaciente» · Entrega y rúbrica

TABLA A · LOS FICHEROS
qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el programa) y **cuáles se suben y adónde**.
TABLA B · LOS REQUISITOS
qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**
EN TOTAL, AL FINAL DEL SPRINT SE SUBEN A `SPRINT-06/`
3 ficheros del equipo (`informe_calidad.md`, `reto_S06.ipynb`, `README.md`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **3 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 6). «El reto» no es un fichero aparte: son esos tres ficheros. Además, el CSV va a `data/` si aún no está. La *review* y la *retro* se hacen de palabra: no se suben.

Qué se sube y cómo se evalúa. Se lee **una vez**, antes de entregar, y se vuelve a la rúbrica cuando el equipo revisa su trabajo. Mientras trabajáis no hace falta: para eso está el reto en una página.

- Volver al README del sprint · ficha del reto, para consultar.
- Vuestra hoja personal: autoevaluación y coevaluación.
- Cómo subir a GitHub paso a paso, con capturas: Cómo entregar en GitHub. Esta hoja responde a **dos preguntas distintas**, una tabla para cada una:

1 · Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-06/`**, salvo el CSV.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`informe_calidad.md`	**Lo creáis a partir de una plantilla.** Os damos `informe_calidad_plantilla.md` con los apartados y la tabla de defectos	Copiar el texto de la plantilla en un archivo nuevo llamado `informe_calidad.md` y rellenarlo, incluida la sección «Revisión con la rúbrica»	**Sí** · `sprint-06/`
`reto_S06.ipynb`	**Lo creáis.** No hay plantilla: abrís un cuaderno nuevo en Colab y reutilizáis el código de `S06_01` a `S06_04`	Inventario de defectos, función `limpiar_reservas`, partición *train/test*, *pipeline*, *baseline* y validación cruzada, en este orden	**Sí** · `sprint-06/`
`README.md`	**Lo modificáis.** El repositorio del equipo ya trae un `README.md` en `sprint-06/` (si no está, lo creáis)	Rellenarlo: cómo ejecutar el cuaderno, versiones de las librerías, quién hizo qué y uso de la IA	**Sí** · `sprint-06/`
`informe_calidad_plantilla.md`	**Ya hecho.** Es solo el punto de partida del informe	Nada: se copia su contenido en `informe_calidad.md`	**No**: se sube ya rellena, con el nombre `informe_calidad.md`
`reservas_sucio.csv`	**Ya hecho.** Los datos del cliente (sintéticos)	Nada: **no se edita a mano**; toda corrección va en el código	**Sí**, sin cambios · `data/` (si aún no está)
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-06/`

2 · Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Tabla de defectos	Al menos **6 defectos**, cada uno con qué es, dónde está, cuántas filas afecta, causa probable, tratamiento y por qué. Los **tres más graves, destacados**	Mínimo	`informe_calidad.md` · apartado 2

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Función de limpieza	**Una sola** función `limpiar_reservas(df)` con docstring que devuelve la tabla limpia **y un resumen de lo que ha cambiado**. No toca el CSV original y no descarta ninguna fila sin contarla y justificarla	Mínimo	`reto_S06.ipynb` y `informe_calidad.md` · apartado 3
Cuaderno que se ejecuta entero	*Entorno de ejecución → Reiniciar y ejecutar todo* termina sin errores	Mínimo	`reto_S06.ipynb`
README	Cómo ejecutar el cuaderno y en qué orden, y quién hizo qué. En el mínimo, versión corta	Mínimo	`README.md`
Entrega en el repositorio	Los tres ficheros en `sprint-06/`, subidos desde el navegador por personas distintas (cada subida es un *commit* con su nombre)	Mínimo	Carpeta `sprint-06/` y su historial
Pipeline sin fuga de datos	Partición *train/test* y `Pipeline` con `ColumnTransformer` que **imputa, escala y codifica ajustando solo con *train***. La función de limpieza solo aplica reglas fijas (no calcula medias ni medianas)	Estándar	`reto_S06.ipynb`
Baseline y modelo comparados	`DummyClassifier` y un modelo (por ejemplo, regresión logística) con validación cruzada, y una frase con **media ± desviación** de la métrica elegida	Estándar	`reto_S06.ipynb` y `informe_calidad.md` · apartado 4
Preguntas al cliente	Al menos 3 preguntas sobre lo que los datos no permiten decidir (por ejemplo, qué significa un precio de 0 €)	Estándar	`informe_calidad.md` · apartado 5
IA responsable	Respuesta concreta a «¿qué pasa si el país de origen se imputa mal? ¿podría el modelo discriminar por nacionalidad? ¿hay columnas que no deberían usarse?» y, en el README, qué se pidió a la IA y qué se revisó	Estándar	`informe_calidad.md` · apartado 6 y `README.md`
README con versiones	Versión de Python y de `pandas` y `scikit-learn`, y el orden de ejecución	Estándar	`README.md`
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa informe, cuaderno y README con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el informe; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`informe_calidad.md` (sección «Revisión con la rúbrica») y `sprint-06/`
Extras	Tabla con 8–10 defectos · partición temporal en lugar de aleatoria, justificada · explicar (o probar) qué pasaría si se ajustara el escalado con todos los datos	Extra	`informe_calidad.md` y `reto_S06.ipynb`

3 · Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay código, pero el cuaderno falla al ejecutarlo entero o se ha corregido el CSV a mano	`limpiar_reservas` es una sola función con docstring, devuelve tabla limpia y resumen, y trata en código todos los defectos de la tabla	Además, el `Pipeline` imputa, escala y codifica **ajustando solo con *train***, y el baseline y el modelo se comparan con validación cruzada	Además, partición temporal justificada o una prueba de qué cambia si se ajusta con todos los datos
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona si se ejecutan las celdas sueltas en cierto orden	*Reiniciar y ejecutar todo* funciona y el README dice cómo ejecutarlo, pero faltan versiones o el orden	Otra persona obtiene **las mismas cifras** siguiendo solo el README, con versiones de las librerías y semilla fijada	Además, el README indica Python, librerías, ficheros necesarios y de dónde sale el CSV, y se ha probado en un cuaderno nuevo de Colab

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	El informe dice «hay nulos» sin más: no dice dónde, cuántas filas ni por qué	Tabla de al menos 6 defectos con dónde, cuántas filas, causa probable y tratamiento, y los tres más graves destacados	Además, cada tratamiento está justificado con su riesgo, hay resumen para el gerente sin jerga, 3 preguntas al cliente y una frase del baseline con media ± desviación	Además, 8–10 defectos y una recomendación para el cliente con los límites del modelo explicados
IA responsable (¿se sabe qué hizo la IA y qué comprobasteis?)	No se menciona la IA ni el riesgo de usar el país de origen	Dice que se ha usado IA, o menciona un riesgo, pero en genérico	Responde con ejemplos del dataset a la pregunta del país de origen y de las columnas que no deben usarse; dice **qué se pidió a la IA, qué se revisó y qué se cambió**	Además, aplica una medida y la evalúa (por ejemplo, el modelo sin `pais_origen` frente al modelo con él) y cuenta un fallo concreto de la IA que detectasteis

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar** `reto_S06.ipynb` con *Entorno de ejecución → Reiniciar y ejecutar todo*, siguiendo solo lo que dice el README. Anotar si algo falla y qué cifras salen.
2. **Leer el informe como si fuera el gerente**: ¿se entiende qué está mal en los datos, cuántas filas afecta y qué se ha hecho con cada defecto? Subrayar lo que no se entienda.
3. **Mirar el código**: `¿limpiar_reservas` es una sola función con docstring? ¿Algún `fit` o `fit_transform` usa datos de *test* o la tabla completa? ¿Hay algo que no sepáis explicar? Que quien lo escribió lo explique.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · el pipeline se ajusta solo con train y hay media ± desviación, pero falta la versión de scikit-learn en el README»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `informe_calidad.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.